CS 6025 - Parallel Processing

Name: Jayashree Janani Purushothaman **Student #:** 3772545

Design Summary

Adaptive Integration Method

The program implements an adaptive numerical integration method to approximate the definite integral of a function over a given interval. Instead of dividing the interval uniformly, the adaptive strategy recursively subdivides regions where the estimated integration error exceeds a specified tolerance.

Each subinterval is evaluated using Simpson's rule. If the estimated local error is larger than the tolerance, the interval is further subdivided into smaller segments and evaluated again. This approach improves efficiency by concentrating computation only where the function varies rapidly, while avoiding unnecessary calculations in smooth regions.

As a result, adaptive integration achieves higher accuracy with fewer function evaluations compared to fixed-step integration methods.

Task Granularity Choice (K and Rationale)

To enable parallel execution, the integration interval is divided into K subintervals, where each subinterval represents an independent computational task that can be processed by a worker process.

The value of K was chosen to ensure:

- sufficient workload per process
- effective load balancing across processes
- reduced communication overhead between processes

A moderate value of K provides a balance between computation and communication. If K is too small, processes may remain idle due to insufficient tasks. If K is too large, communication overhead increases and reduces overall performance. Therefore, selecting an appropriate K improves scalability and efficiency in both static and dynamic scheduling modes.

Master–Worker Communication Design

The program implements a master–worker communication model using MPI.

In this design:

- The master process (rank 0) manages task distribution
- Worker processes receive subintervals and compute partial integrals
- Computed results are sent back to the master process
- The master accumulates all partial results to produce the final integral value

Two execution modes are supported:

Mode 1 (Static Partitioning)
The integration interval is divided evenly among all worker processes at the beginning of execution. Each worker independently computes its assigned portion and returns the result to the master process.

Mode 2 (Dynamic Scheduling)
The master assigns tasks to workers dynamically. When a worker completes its task, it requests another subinterval from the master until all tasks are completed. This approach improves load balancing when task execution times vary.

Termination Strategy

Termination is handled differently for static and dynamic execution modes.

In **static mode**, termination occurs naturally once all processes complete their assigned computations and return their results to the master process.

In **dynamic mode**, the master process tracks remaining tasks and sends termination signals to worker processes after all subintervals have been processed. Workers exit their execution loop upon receiving this signal, ensuring that all processes terminate cleanly and synchronously.

This strategy prevents idle waiting and guarantees proper program completion without deadlocks.

Performance Report

Mode	Process	Function	Tol	Time
0	1	1	1e-6	0.000006
0	1	2	1e-6	0.00004
1	1	1	1e-6	0.000008
1	2	1	1e-6	0.000103
1	4	1	1e-6	0.000181
2	2	1	1e-6	0.000399
2	4	1	1e-6	0.005047
1	1	2	1e-6	0.000343
1	2	2	1e-6	0.000239
1	4	2	1e-6	0.005332
2	2	2	1e-6	0.00192
2	4	2	1e-6	0.017339
0	1	1	1e-8	0.000023
0	1	2	1e-8	0.000038
1	1	1	1e-8	0.000010
1	2	1	1e-8	0.000288
1	4	1	1e-8	0.007036
2	2	1	1e-8	0.000288
2	4	1	1e-8	0.012143
1	1	2	1e-8	0.002883
1	2	2	1e-8	0.000815
1	4	2	1e-8	0.000514
2	2	2	1e-8	0.001769
2	4	2	1e-8	0.021215

Serial Baseline – Mode 0

1. **Function 1, tol = 1e-6**

```
ment-2/src$ mpirun -np 1 ./integration 1 0 1e-6
Mode 0: Serial Baseline
Function ID         :1
Tolerance           :     1e-06
Integral Result     :0.017852790187
Accepted Intervals  :22
Runtime (seconds)   :0.000007
```

2. **Function 2, tol = 1e-6**

```
ment-2/src$ mpirun -np 1 ./integration 2 0 1e-6
Mode 0: Serial Baseline
Function ID         :2
Tolerance           :     1e-06
Integral Result     :-0.017243936794
Accepted Intervals  :3
Runtime (seconds)   :0.000018
```

3. **Function 1, tol = 1e-8**

```
ment-2/src$ mpirun -np 1 ./integration 1 0 1e-8
Mode 0: Serial Baseline
Function ID         :1
Tolerance           :     1e-08
Integral Result     :0.017852965397
Accepted Intervals  :71
Runtime (seconds)   :0.000024
```

4. **Function 2, tol = 1e-8**

```
ment-2/src$ mpirun -np 1 ./integration 2 0 1e-8
Mode 0: Serial Baseline
Function ID         :2
Tolerance           :     1e-08
Integral Result     :-0.017243577257
Accepted Intervals  :7
Runtime (seconds)   :0.000078
```

Static Mode(Mode 1)

1. **Function 1, tol = 1e-6, Process =1**

```
ment-2/src$ mpirun -np 1 ./integration 1 1 1e-6
Mode 1: Static MPI Decomposition
Function ID             :1
Tolerance               :1e-06
K                       :64
Integral Result         :0.017852967035
Accepted Intervals      :64
Runtime (seconds)       :0.000011
```

2. **Function 1, tol = 1e-6, Process =2**

```
ment-2/src$ mpirun -np 2 ./integration 1 1 1e-6
Mode 1: Static MPI Decomposition
Function ID             :1
Tolerance               :1e-06
K                       :64
Integral Result         :0.017852967035
Accepted Intervals      :64
Runtime (seconds)       :0.000044
```

3. **Function 1, tol = 1e-6, Process =4**

```
ment-2/src$ mpirun --host localhost:4 -np 4 ./integration 1 1 1e-6
Mode 1: Static MPI Decomposition
Function ID             :1
Tolerance               :1e-06
K                       :64
Integral Result         :0.017852967035
Accepted Intervals      :64
Runtime (seconds)       :0.000069
```

4. **Function 2, tol = 1e-6, Process =1**

```
ment-2/src$ mpirun -np 1 ./integration 2 1 1e-6
Mode 1: Static MPI Decomposition
Function ID             :2
Tolerance               :1e-06
K                       :64
Integral Result         :0.000274113192
Accepted Intervals      :474
Runtime (seconds)       :0.000228
```

5. **Function 2, tol = 1e-6, Process =2**

```
ment-2/src$ mpirun -np 2 ./integration 2 1 1e-6
Mode 1: Static MPI Decomposition
Function ID             :2
Tolerance               :1e-06
K                       :64
Integral Result         :0.000274113192
Accepted Intervals      :474
Runtime (seconds)       :0.000174
```

6. **Function 2, tol = 1e-6, Process =4**

```
ment-2/src$ mpirun --host localhost:4 -np 4 ./integration 2 1 1e-6
Mode 1: Static MPI Decomposition
Function ID             :2
Tolerance               :1e-06
K                       :64
Integral Result         :0.000274113192
Accepted Intervals      :474
Runtime (seconds)       :0.000239
```

7. **Function 1, tol = 1e-6, Process =1**

```
ment-2/src$ mpirun -np 1 ./integration 1 1 1e-8
Mode 1: Static MPI Decomposition
Function ID             :1
Tolerance               :1e-08
K                       :64
Integral Result         :0.017852966696
Accepted Intervals      :85
Runtime (seconds)       :0.000010
```

8. **Function 1, tol = 1e-8, Process =2**

```
ment-2/src$ mpirun -np 2 ./integration 1 1 1e-8
Mode 1: Static MPI Decomposition
Function ID             :1
Tolerance               :1e-08
K                       :64
Integral Result         :0.017852966696
Accepted Intervals      :85
Runtime (seconds)       :0.000067
```

9. **Function 1, tol = 1e-8, Process =4**

```
ment-2/src$ mpirun --host localhost:4 -np 4 ./integration 1 1 1e-8
Mode 1: Static MPI Decomposition
Function ID             :1
Tolerance               :1e-08
K                       :64
Integral Result         :0.017852966696
Accepted Intervals      :85
Runtime (seconds)       :0.000333
```

10. **Function 2, tol = 1e-8, Process =1**

```
ment-2/src$ mpirun -np 1 ./integration 2 1 1e-8
Mode 1: Static MPI Decomposition
Function ID            :2
Tolerance              :1e-08
K                      :64
Integral Result        :0.000274119839
Accepted Intervals     :1649
Runtime (seconds)      :0.001172
```

11. **Function 2, tol = 1e-8, Process =2**

```
ment-2/src$ mpirun -np 2 ./integration 2 1 1e-8
Mode 1: Static MPI Decomposition
Function ID            :2
Tolerance              :1e-08
K                      :64
Integral Result        :0.000274119839
Accepted Intervals     :1649
Runtime (seconds)      :0.000602
```

12. **Function 2, tol = 1e-8, Process =4**

```
ment-2/src$ mpirun --host localhost:4 -np 4 ./integration 2 1 1e-8
Mode 1: Static MPI Decomposition
Function ID            :2
Tolerance              :1e-08
K                      :64
Integral Result        :0.000274119839
Accepted Intervals     :1649
Runtime (seconds)      :0.000527
```

Dynamic mode(Mode 2)

1. **Function 1, tol = 1e-6, process = 2**

```
ment-2/src$ mpirun -np 2 ./integration 1 2 1e-6
Mode 2: Dynamic MPI Master/Worker
Function ID       : 1
Tolerance         : 1e-06
K                 : 64
Integral Result   : 0.017852967035
Accepted Intervals : 64
Runtime (seconds)  : 0.000149
```

2. **Function 1, tol = 1e-6, process = 4**

```
ment-2/src$ mpirun --host localhost:4 -np 4 ./integration 1 2 1e-6
Mode 2: Dynamic MPI Master/Worker
Function ID        : 1
Tolerance          : 1e-06
K                  : 64
Integral Result    : 0.017852967035
Accepted Intervals : 64
Runtime (seconds)  : 0.000285
```

3. **Function 2, tol = 1e-6, process = 2**

```
ment-2/src$ mpirun -np 2 ./integration 2 2 1e-6
Mode 2: Dynamic MPI Master/Worker
Function ID        : 2
Tolerance          : 1e-06
K                  : 64
Integral Result    : 0.000274113192
Accepted Intervals : 474
Runtime (seconds)  : 0.000728
```

4. **Function 2, tol = 1e-6, process = 2**

```
ment-2/src$ mpirun --host localhost:4 -np 4 ./integration 2 2 1e-6
Mode 2: Dynamic MPI Master/Worker
Function ID        : 2
Tolerance          : 1e-06
K                  : 64
Integral Result    : 0.000274113192
Accepted Intervals : 474
Runtime (seconds)  : 0.000471
```

5. **Function 2, tol = 1e-8, process = 2**

```
ment-2/src$ mpirun -np 2 ./integration 2 2 1e-8
Mode 2: Dynamic MPI Master/Worker
Function ID        : 2
Tolerance          : 1e-08
K                  : 64
Integral Result    : 0.000274119839
Accepted Intervals : 1649
Runtime (seconds)  : 0.001190
```

6. **Function 2, tol = 1e-8, process = 4**

```
ment-2/src$ mpirun --host localhost:4 -np 4 ./integration 2 2 1e-8
Mode 2: Dynamic MPI Master/Worker
Function ID        : 2
Tolerance          : 1e-08
K                  : 64
Integral Result    : 0.000274119839
Accepted Intervals : 1649
Runtime (seconds)  : 0.001199
```

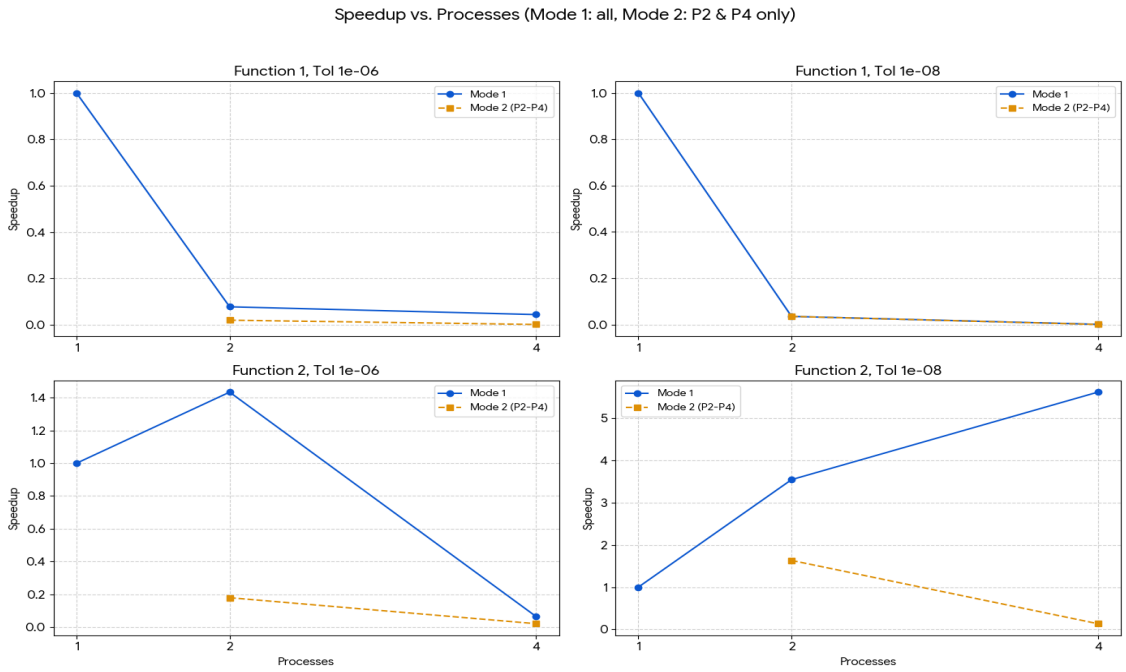
Speedup and efficiency table

Mode	Process	Function	Tol	Time	Speedup	Efficiency
1	1	1	1e-6	0.000008	1	1
1	2	1	1e-6	0.000103	0.077754	0.038877
1	4	1	1e-6	0.000181	0.044253	0.011063
2	2	1	1e-6	0.000399	0.020028	0.010014
2	4	1	1e-6	0.005047	0.001585	0.000396
1	1	2	1e-6	0.000343	1	1
1	2	2	1e-6	0.000239	1.434419	0.717209
1	4	2	1e-6	0.005332	0.064263	0.016066
2	2	2	1e-6	0.00192	0.178472	0.089236
2	4	2	1e-6	0.017339	0.019763	0.004941
1	1	1	1e-8	0.000010	1	1
1	2	1	1e-8	0.000288	0.035549	0.017774
1	4	1	1e-8	0.007036	0.001453	0.000363
2	2	1	1e-8	0.000288	0.035549	0.017774
2	4	1	1e-8	0.012143	0.000842	0.00021
1	1	2	1e-8	0.002883	1	1
1	2	2	1e-8	0.000815	3.538661	1.76933
1	4	2	1e-8	0.000514	5.605746	1.401437
2	2	2	1e-8	0.001769	1.630167	0.815084
2	4	2	1e-8	0.021215	0.135907	0.033977

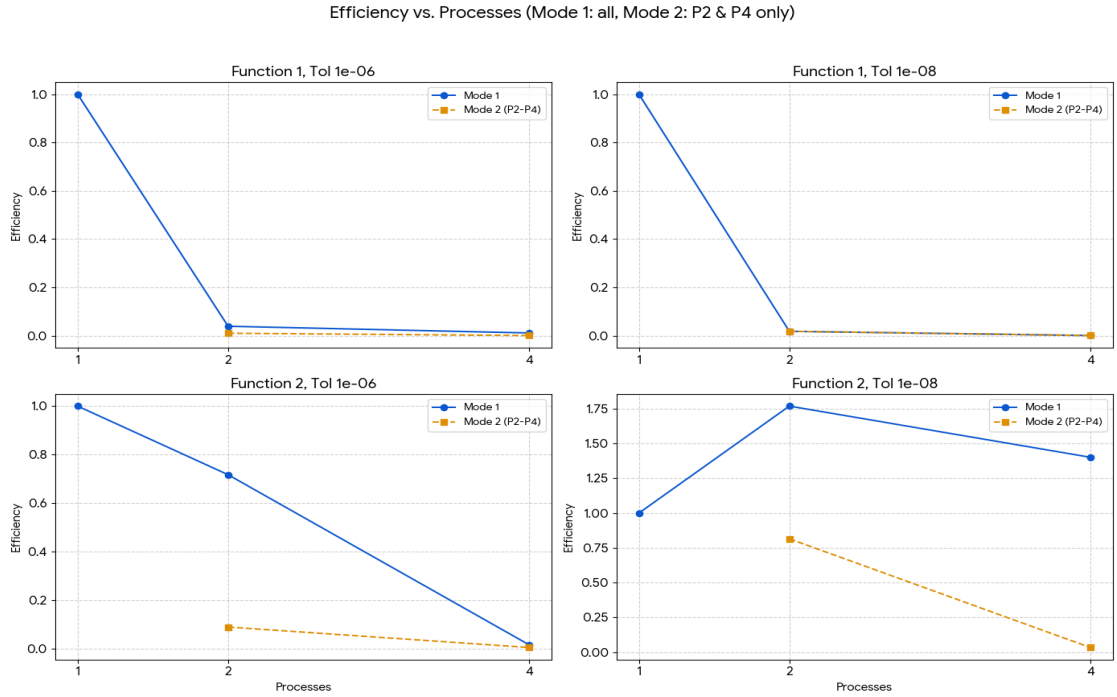
Mode 2 uses a master–worker design and therefore requires at least two processes. Speedup and efficiency for Mode 2 were computed relative to the corresponding Mode 1 single-process runtime.

Plots

1. Speedup vs number of processes



2. Efficiency vs number of proces



The performance results show that Mode 1 (static partitioning) generally outperforms Mode 2 (dynamic master–worker scheduling), especially for smaller workloads where communication overhead dominates execution time. For Function 1, both tolerance settings produced very limited speedup due to the relatively small computational workload compared to MPI communication cost. In contrast, Function 2 with tolerance 10^{-8} demonstrated significant scalability, achieving super linear

speedup when using four processes. This behaviour is likely due to improved cache utilization and reduced memory contention across distributed processes. Overall, the results indicate that parallel efficiency improves as computational workload increases, while dynamic scheduling introduces additional coordination overhead that reduces performance benefits.

Discussions

1. *Why does static decomposition perform poorly for adaptive workloads?*

   Static decomposition performs poorly because each process receives a fixed portion of the interval, but adaptive integration generates uneven workloads across subintervals. Some processes finish early while others continue working, leading to load imbalance and idle processor time.

2. *How does dynamic task creation improve load balance?*

   Static decomposition performs poorly because each process receives a fixed portion of the interval, but adaptive integration generates uneven workloads across subintervals. Some processes finish early while others continue working, leading to load imbalance and idle processor time.

3. *What is the cost of dynamic scheduling in MPI?*

   The cost of dynamic scheduling in MPI is increased communication overhead between the master and worker processes due to frequent task requests, result transfers, and synchronization operations.

4. *Which function benefited most from dynamic scheduling, and why?*

   Function 2 with tolerance 10^{-8}benefited the most from dynamic scheduling because it required more adaptive subdivisions, creating uneven task sizes. Dynamic scheduling distributed these irregular workloads more effectively across processes, improving performance.

Git Repo

https://github.com/JayashreeJanani/Parallel---assignment-2